

모듈 개요

- 모듈이란?

함수, 변수, 클래스의 집합. 함수, 변수, 클래스들을 모아놓은 파일

여러 변수, 함수, 클래스(다수의 변수+함수)를 저장해둔 파이썬 소스코드 파일.

*.py 확장자를 이용하여 직접 만들 수 있다.

다른 사람들에 의해서 만들어진 파이썬 라이브러리들을 모듈이라고 한다.

모듈을 불러오는 방법 : import 명령을 통해서 모듈을 불러와 사용할 수 있다.

- 모듈의 종류

- 표준 모듈 : 파이썬이 기본적으로 제공해주는 모듈

- 외부 모듈 : 파이썬이 기본적으로 제공해주지 않는 모듈

- 사용자정의 모듈 : 직접 만들어 사용하는 모듈

모듈 import

- 모듈을 사용하기 위해서는 다음과 같은 문법을 이용하여 모듈을 임포트 해야 한다.

```
import <모듈이름>
```

```
import <모듈이름> as <모듈변수>
```

```
from <모듈> import <모듈이름>,<모듈이름>
```

```
from 모듈이름 import 모듈함수1, 모듈함수2, ....
```

from ~ import ~ 문을 이용하면 모듈이름을 붙이지 않고 바로 해당 모듈 함수 사용 가능
(콤마)를 사용하지 않고 여러 함수를 불러올 수 있는 방법

```
from 모듈이름 import *
```

math 모듈

- 수학 함수를 사용할 수 있는 math 모듈

```
import math
```

```
import math
```

```
help(math)
```

```
print(math.sin(90))
```

```
print(dir(math))
```

```
print(math.pi)
```

```
math.__doc__
```

```
import math as m
```

```
from math import sin, pi
```

```
print(m.sin(90))
```

```
print(sin(90))
```

```
print(m.pi)
```

```
print(pi)
```

math 모듈

- 수평선 상에서의 데이터 움직임과 관련된 수학 함수들
 - 올림 (`math.ceil()`): 무조건 오른쪽 방향에 있는 가장 가까운 정수로 이동
 - 내림 (`math.floor()`): 무조건 왼쪽 방향에 있는 가장 가까운 정수로 이동
 - 반올림 (`round()`)

```
print(math.floor(10.678899), round(10.678899), math.ceil(10.2345))
```

```
# 10 11 11
```

math 모듈

소수점 특정 자릿수 제어 실습

```
pi_value = 3.141592
```

```
ceil_value = math.ceil(pi_value * 100) / 100
```

```
print(f"[1] 소수점 둘째 자리까지 올림: {ceil_value}") # 출력: 3.15
```

```
floor_value = math.floor(pi_value * 100) / 100
```

```
print(f"[2] 소수점 둘째 자리까지 내림: {floor_value}") # 출력: 3.14
```

math 모듈

```
print("=== 팩토리얼 (Factorial) ===")
```

```
students = ["A", "B", "C"]
```

```
# 3명의 학생을 일렬로 세우는 경우의 수 ( $3! = 3 * 2 * 1$ )
```

```
cases = math.factorial(len(students))
```

```
print(f"학생 목록: {students}")
```

```
print(f"🎯 3명을 일렬로 세우는 총 경우의 수: {cases}가지")
```

random 모듈

`random.randint(start, end)` : start~end 사이의 정수 난수

`random.randrange(n)` : 0~n 까지의 정수 추출

`random.random()` : 0.0 ~1.0 사이의 실수 값

`random.uniform(n1,n2)` : n1~n2 사이의 실수

`random.choice(리스트)` : 리스트에서 랜덤하게 뽑는다.

`random.shuffle(리스트)` : 리스트를 랜덤하게 섞는다.

`random.sample(시퀀스데이터, k)` : 시퀀스데이터에서 k개를 추출한다

```
import random
print(dir(random))
help(random)
```

random 모듈

```
import random
print(random.randint(1,5))
print([random.randint(0,1) for i in range(10)])
print([random.randrange(11) for i in range(10)])
print([random.random() for i in range(3)])
print([round(random.random(),2) for i in range(3)])
print([round(random.uniform(-5,5),1) for i in range(10)])
```

```
2
[0, 1, 1, 1, 0, 1, 1, 1, 0, 1]
[3, 9, 6, 7, 6, 6, 4, 4, 6, 2]
[0.5390256523819442, 0.21447748061317684, 0.03384613929261193]
[0.5, 0.52, 0.97]
[3.5, -2.3, 3.2, -1.9, 1.5, -1.6, -0.7, -0.9, -2.1, 3.9]
```


random 모듈

```
print(random.sample(range(100),10))
```

```
print(random.sample('도 레 미 파 솔 라 시 도'.split(),3))
```

```
student_list = ['철수', '영희', '길동', '미미', '영철', '은정', '흥민']
```

```
print(random.sample(student_list , 3))
```

```
[23, 81, 75, 94, 62, 66, 21, 27, 26, 0]  
['파', '도', '미']  
['은정', '미미', '길동']
```

random 모듈

리스트 요소 섞기

```
student_list = ['철수', '영희', '길동', '미미', '영철', '은정', '홍민']
```

```
print('Before : ', student_list)
```

```
random.shuffle(student_list)
```

```
print('After : ', student_list)
```

리스트에서 한개 뽑기

```
print(random.choice(student_list))
```

```
print(random.choice(student_list))
```

```
print(random.choice(student_list))
```

홍민
영희
철수

Before : ['철수', '영희', '길동', '미미', '영철', '은정', '홍민']

After : ['길동', '철수', '은정', '미미', '영희', '홍민', '영철']

random 모듈

로또 번호 뽑기 - 중복있음

1 ~ 45 범위에서 6개 추출한 후 리스트로 생성

```
def make_lotto1():
```

```
    lotto_list = []
```

```
    for i in range(6):
```

```
        temp = random.randint(1,45)
```

```
        lotto_list.append(temp)
```

```
    return sorted(lotto_list)
```

[12, 13, 23, 24, 27, 31]

[1, 2, 13, 15, 36, 40]

[19, 25, 26, 38, 38, 39]

```
print(make_lotto1())
```

```
print(make_lotto1())
```

```
print(make_lotto1())
```

random 모듈

리스트 내포방식 활용

로또 번호 뽑기 - 중복있음 + 오름차순 정렬

```
print(sorted([random.randint(1,45) for i in range(6)]))
```

```
print(sorted([random.randint(1,45) for i in range(6)]))
```

```
print(sorted([random.randint(1,45) for i in range(6)]))
```

```
[11, 16, 25, 25, 30, 31]
```

```
[5, 26, 26, 38, 38, 41]
```

```
[5, 6, 11, 14, 26, 34]
```

random 모듈

로또 번호 뽑기 - 중복없음

1 ~ 45 범위에서 6개 추출한 후 리스트로 생성

```
def make_lotto2():
```

```
    lotto_list = []
```

```
    while True:
```

```
        if len(lotto_list) == 6:
```

```
            break
```

```
        temp = random.randint(1, 45)
```

```
        if temp not in lotto_list:
```

```
            lotto_list.append(temp)
```

```
    return sorted(lotto_list)
```

```
print(make_lotto2())
```

```
print(make_lotto2())
```

```
print(make_lotto2())
```

```
[7, 9, 23, 38, 43, 45]
```

```
[10, 11, 18, 25, 31, 39]
```

```
[12, 16, 19, 20, 22, 33]
```

random 모듈

로또 번호 뽑기 - 중복없음

1 ~ 45 범위에서 6개 추출한 후 리스트로 생성

random.sample(리스트, 숫자)

def make_lotto2():

 num_list = list(range(1, 46))

 lotto_list = random.sample(num_list, 6)

 return print(f' 로또 번호 => {lotto_list} ')

print('='*10)

make_lotto2()

make_lotto2()

make_lotto2()

[5, 10, 13, 14, 29, 39]

[1, 15, 29, 35, 44, 45]

[5, 10, 14, 18, 36, 42]

퀴즈

아래의 학생 리스트에서 일주일(월~금) 청소당번 2명을 뽑아서 출력하여라

```
stList = [ '김철수', '홍길동', '기대주', '이동백',  
           '하민수', '김영희', '소민주', '신은수' ]
```

월	청소	당번은?	김영희,	소민주
화	청소	당번은?	김철수,	신은수
수	청소	당번은?	소민주,	이동백
목	청소	당번은?	이동백,	기대주
금	청소	당번은?	홍길동,	기대주

퀴즈

무작위로 퀴즈를 출제하도록 프로그래밍 하여라.

```
quiz_list = [  
    {  
        "question": "파이썬에서 난수를 생성하는 모듈은?",  
        "options": ["1. math", "2. random", "3. time", "4. os"],  
        "answer": 2  
    },  
    {  
        "question": "리스트에서 요소를 무작위로 하나 뽑는 함수는?",  
        "options": ["1. shuffle", "2. randint", "3. choice", "4. sample"],  
        "answer": 3  
    },  
    {  
        "question": "파이썬의 실행 결과값이 항상 짝수로 반올림되는 함수는?",  
        "options": ["1. round", "2. ceil", "3. floor", "4. trunc"],  
        "answer": 1  
    }  
]
```


퀴즈

출력 예시 >>

문제: 파이썬에서 난수를 생성하는 모듈은?

1. math
2. random
3. time
4. Os

정답 번호를 입력하세요: 2
정답입니다!

사용자정의 모듈

- 모듈이란 '함수, 클래스, 변수 등이 정의된 .py 파일로 개발자가 직접 파이썬 함수를 이용하여 코딩하고 파이썬 파일 (*.py)로 저장한다.
- 만든 모듈 불러와서 사용할때에는 import 파이썬파일명 으로 임포트하면 파일내의 함수, 클래스 등을 사용할 수 있다
- **if __name__ == "__main__":**
모듈을 만들 때 이 코드를 넣는 이유는 "이 모듈이 불러갈 때(import)는 동작하지 않고, 직접 실행(Run)할 때만 테스트용으로 출력해주는 안전장치" 라고 할 수 있다.

사용자정의 모듈

```
# 파일명: my_tools.py
def say_hello(name):
    return f"안녕하세요, {name}님! 드림 강사님의 수업에 오신 걸 환영합니다."
def add_bonus(score):
    return score + 10

# 모듈 내부에서 직접 실행할 때만 동작하는 테스트 코드
if __name__ == "__main__":
    print("이것은 my_tools 모듈 내부에서 직접 실행 중입니다.")
    print(say_hello("테스터"))
```

사용자정의 모듈

```
# 파일명: main.py
import my_tools

message = my_tools.say_hello("수강생")
print(message)

new_score = my_tools.add_bonus(85)
print(f"보너스 적용 점수: {new_score}")
```

안녕하세요, 수강생님! 드림 강사님의 수업에 오신 걸 환영합니다.
보너스 적용 점수: 95

퀴즈

모듈을 이용하여 구구단을 출력하도록 프로그래밍 하여라

1단계 - 2개의 함수가 정의된 모듈 만들기 (gugu.py)

`gugu_print1(n)`: 매개변수 `n`을 입력받아 해당 단만 출력하는 함수.

`gugu_print2()`: 전체 구구단(2단~9단)을 출력하는 함수.

모듈 파일 하단에 `if __name__ == "__main__":` 문구를 사용하여, 직접 실행 시에만 "gugu.py 구구단 출력함수 셋팅 완료"라는 문구가 출력되도록 코드 작성

퀴즈

[2 단계] 모듈 불러오기 (main_gugu.py)

gugu 모듈을 가져온다

gugu_print1(n)을 호출하여 n단을 출력한다.

gugu_print2()를 호출하여 전체 구구단을 출력한다.

>> 출력예시1

3 단

3 X 1 = 3

3 X 2 = 6

3 X 3 = 9

3 X 4 = 12

3 X 5 = 15

3 X 6 = 18

3 X 7 = 21

3 X 8 = 24

3 X 9 = 27

퀴즈

>> 출력예시2.

구구단 출력

2 X 1 = 2	3 X 1 = 3	4 X 1 = 4
2 X 2 = 4	3 X 2 = 6	4 X 2 = 8
2 X 3 = 6	3 X 3 = 9	4 X 3 = 12
2 X 4 = 8	3 X 4 = 12	4 X 4 = 16
2 X 5 = 10	3 X 5 = 15	4 X 5 = 20
2 X 6 = 12	3 X 6 = 18	4 X 6 = 24
2 X 7 = 14	3 X 7 = 21	4 X 7 = 28
2 X 8 = 16	3 X 8 = 24	4 X 8 = 32
2 X 9 = 18	3 X 9 = 27	4 X 9 = 36

5 X 1 = 5	6 X 1 = 6	7 X 1 = 7
5 X 2 = 10	6 X 2 = 12	7 X 2 = 14
5 X 3 = 15	6 X 3 = 18	7 X 3 = 21
5 X 4 = 20	6 X 4 = 24	7 X 4 = 28
5 X 5 = 25	6 X 5 = 30	7 X 5 = 35
5 X 6 = 30	6 X 6 = 36	7 X 6 = 42
5 X 7 = 35	6 X 7 = 42	7 X 7 = 49
5 X 8 = 40	6 X 8 = 48	7 X 8 = 56
5 X 9 = 45	6 X 9 = 54	7 X 9 = 63

8 X 1 = 8	9 X 1 = 9
8 X 2 = 16	9 X 2 = 18
8 X 3 = 24	9 X 3 = 27
8 X 4 = 32	9 X 4 = 36
8 X 5 = 40	9 X 5 = 45
8 X 6 = 48	9 X 6 = 54
8 X 7 = 56	9 X 7 = 63
8 X 8 = 64	9 X 8 = 72
8 X 9 = 72	9 X 9 = 81

오류 발생 알아보기

- 오류는 어떤 때 발생하는가?

NameError: 함수이름, 변수, 리스트 이름등이 잘못된 경우

IndexError : 튜플, 리스트의 잘못된 인덱스 접근

ZeroDivisionError : 0으로 나눈 경우

FileNotFoundError : 잘못된 파일 경로

TypeError : 자료형 에러

ValueError : 리터럴값 에러

SyntaxError 제외 => 예외처리 try: ~ Except 구문에서 제외

오류발생 알아보기

```
1 print(12/0)
```

```
ZeroDivisionError                                Traceback (most recent call last)
<ipython-input-59-bb239b366562> in <module>
----> 1 print(12/0)
```

ZeroDivisionError: division by zero

```
1 f = open("나없는파일", 'r')
```

```
FileNotFoundError                                Traceback (most recent call last)
<ipython-input-60-9c14a73c8317> in <module>
----> 1 f = open("나없는파일", 'r')
```

FileNotFoundError: [Errno 2] No such file or directory: '나없는파일'

오류발생 알아보기

```
1 fruit = ['사과', '포도', '배']  
2 fruit[3]
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-61-22b147da8cfd> in <module>  
      1 fruit = ['사과', '포도', '배']  
----> 2 fruit[3]
```

IndexError: list index out of range

```
1 int('일백')
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-62-b78511d7a942> in <module>  
----> 1 int('일백')
```

ValueError: invalid literal for int() with base 10: '일백'

오류발생 알아보기

```
1 print(user)
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-63-9a48b9113df2> in <module>  
----> 1 print(user)
```

NameError: name 'user' is not defined

```
1 ','.join(range(10))
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-64-dac9d57e9905> in <module>  
----> 1 ','.join(range(10))
```

TypeError: sequence item 0: expected str instance, int found

예외처리

- 예외(Exception)이란?

오류가 발생을 하면 메시지를 출력하거나 오류를 무시하는 기능.

프로그램 처리상 에러 발생시 예외 상황을 처리해 주는 것을 예외 처리라고 한다.

- 예외처리 문법

try..except 명령

try..except..else 명령

try..except..else..finally 명령

raise Exception : 사용자정의 에러

try.. except..문

- try ... except : 발생한 오류를 메시지로 출력

try :
 명령어
except 에러코드 (as e) :
 에러처리명령

try:

```
print(12/7)
```

```
print(12/0)
```

```
except ZeroDivisionError as e:
```

```
    print(f'0으로 나눌때 발생하는 에러입니다. => {e}')
```

```
print('예외처리 테스트 종료')
```

```
1.7142857142857142
```

```
0으로 나눌때 발생하는 에러입니다. => division by zero
```

```
예외처리 테스트 종료
```

try.. except..문

```
result = []
for i in range(3):
    try:
        temp = int(input('데이타 입력 => '))
        result.append(temp)
    except ValueError as e:
        print('숫자가 아닙니다. 에러 발생', e)
print(result)
```

```
데이타 입력 => 45
데이타 입력 => 강아지
숫자가 아닙니다. 에러 발생 invalid literal for int() with base 10: '강아지'
데이타 입력 => -90
[45, -90]
```

try.. except..문

except 구문이 여러개인 경우 함수 정의

```
def div_except(x, y):
```

```
    try:
```

```
        print(x/y)
```

```
    except ZeroDivisionError as e:
```

```
        print(f'0으로 나눌때 발생하는 에러입니다. => {e}')
```

```
    except TypeError as e:
```

```
        print(f'TypeError => {e}')
```

```
    print('예외처리 테스트 종료\n\n')
```

함수 호출

```
div_except(12, 3)
```

```
div_except(12, 0)
```

```
div_except('a', 'b')
```

4.0

예외처리 테스트 종료

0으로 나눌때 발생하는 에러입니다. => division by zero

예외처리 테스트 종료

TypeError => unsupported operand type(s) for /: 'str' and 'str'

예외처리 테스트 종료

Exception 을 이용한 try.. except..문

모든 예외의 에러 메시지를 출력할 때는 Exception 키워드
에러코드를 몰라도 된다. => Exception
e는 에러메세지

try:

 명령

except Exception as e:

 print(e)

Exception 을 이용한 try.. except..문

```
def exceptionTest(x,y):
```

```
    try:
```

```
        print(x/y)
```

```
    except Exception as e:
```

```
        print(f'에러 발생 => {e}')
```

```
    print('예외처리 테스트 종료')
```

```
exceptionTest('ten', 'five')
```

```
exceptionTest(10, 0)
```

```
exceptionTest(10, 5)
```

```
에러 발생 => unsupported operand type(s) for /: 'str' and 'str'  
예외처리 테스트 종료
```

```
에러 발생 => division by zero  
예외처리 테스트 종료
```

```
2.0  
예외처리 테스트 종료
```

pass + try.. except..문

- try .. except : 발생한 오류 무시. pass 문 이용

```
numList = [-42, '21', '십오', 39, -66, '구십삼', -38, 15, 18, 63]
```

```
for item in numList:
```

```
    try:
```

```
        print(abs(item))
```

```
    except Exception as e:
```

```
        pass
```

42

39

66

38

15

18

63

try .. except...else 문

try:

 명령어

except 에러코드 as e:

except Exception as e:

except:

 e 출력 ,에러처리명령

else:

 에러가 발생하기 않은 경우 명령어

```
def exceptionTest2(x,y):
```

```
    try:
```

```
        print(f'x = {x} y = {y} , {x} / {y} = {x/y:.2f}')
```

```
    except Exception as e:
```

```
        print('에러발생 => ', e)
```

```
    else:
```

```
        print('에러가 검출되지 않았습니다')
```

```
    print('예외처리 테스트 종료\n')
```

try .. except...else 문

```
exceptionTest2('ten', 'five')
```

```
exceptionTest2(10, 0)
```

```
exceptionTest2('삼', 10)
```

```
exceptionTest2(100, 200)
```

에러발생 => unsupported operand type(s) for /: 'str' and 'str'
예외처리 테스트 종료

에러발생 => division by zero
예외처리 테스트 종료

에러발생 => unsupported operand type(s) for /: 'str' and 'int'
예외처리 테스트 종료

x = 100 y = 200 , 100 / 200 = 0.50
에러가 검출되지 않았습니다
예외처리 테스트 종료

퀴즈

입력된 글자가 숫자가 아니라면 에러 메시지 출력, 입력된 글자가 모두 숫자라면 두 수를 더한 후 출력한다.

```
1 add_except()
```

숫자1 입력 ? 백
입력 데이터가 숫자가 아닙니다.

```
1 add_except()
```

숫자1 입력 ? -90
숫자2 입력 ? 89
 $-90 + 89 = -1$

```
1 add_except()
```

숫자1 입력 ? 100
숫자2 입력 ? 강아지
입력 데이터가 숫자가 아닙니다.

퀴즈

문자열 데이터 리스트에서 숫자 데이터만 별도 정수 형태로 변경한 후 새로운 리스트에 추가하여 결과를 출력하는 함수를 프로그래밍 하시오

- try except 예외처리 구문을 이용한다.
- 정수 형태로 변경할 수 없는 데이터의 경우 메시지를 띄우는 대신 아무런 작업도 수행하지 않도록 pass 키워드를 이용한다.

변환된 숫자 리스트: [10, 20, 40] 길이: 3

변환된 숫자 리스트: [327, -100, 0, -890] 길이: 4

퀴즈

>> 함수 호출 예시

```
numbers_list = make_int_list(["10", "20", "abc", "40"])  
print(f"변환된 숫자 리스트: {numbers_list} {type(numbers_list)}\n")
```

```
numbers_list = make_int_list(["327", "파이썬", "-100", "123-890", "0", "3.14"])  
print(f"변환된 숫자 리스트: {numbers_list} {type(numbers_list)}\n")
```

try..except..else..finally 문

try:

 명령어

except 에러코드 as e:

except Exception as e:

except:

 print(e)

 에러처리명령

else:

 에러가 발생하기 않은 경우 명령어

finally:

 무조건 실행되는 명령어

```
def divide_except(x, y):
```

```
    try:
```

```
        result = x/y
```

```
    except Exception as e:
```

```
        print('에러발생 => ', e)
```

```
    else:
```

```
        print(f'{x} 나누기 {y} 는(은) {result} ')
```

```
    finally:
```

```
        print('예외처리 테스트 종료')
```


try..except..else..finally 문

```
divide_except(100, 200)
```

```
divide_except('삼', 10)
```

```
divide_except(10,0)
```

100 나누기 200 는(은) 0.5

예외처리 테스트 종료

에러발생 => unsupported operand type(s) for /: 'str' and 'int'

예외처리 테스트 종료

에러발생 => division by zero

예외처리 테스트 종료

퀴즈

회원의 이메일 주소와 나이 데이터를 함수로 전달시켜 이메일 형식과 정수 형태의 데이터가 맞다면 특정 메시지가 출력되도록 프로그래밍 하시오.

- try except else finally 예외처리 구문을 이용한다.
- 이메일 데이터의 유효성 검사는 index() 문자열 함수를 이용한다.
- 에러 메시지는 as e 구문을 이용하여 출력한다.

>> 함수 호출 예시

```
register_user("dream@python.com", "25")  
register_user("wrong_email.com", "30")  
register_user("test@test.com", "twenty")
```

퀴즈

>> 결과 예시

```
--- [회원 등록 시도: dream@python.com] ---  
등록 성공! [이메일: dream@python.com, 나이: 25세]  
--- [회원가입 절차 완료] ---
```

```
--- [회원 등록 시도: wrong_email.com] ---  
등록 실패: 입력 데이터 형식이 올바르지 않습니다.  
(substring not found)  
--- [회원가입 절차 완료] ---
```

```
--- [회원 등록 시도: test@test.com] ---  
등록 실패: 입력 데이터 형식이 올바르지 않습니다. (invalid  
literal for int() with base 10: 'twenty')  
--- [회원가입 절차 완료] ---
```

파일 읽고 쓰기

- 파이썬의 File I/O

```
f = open( " 파일경로 " , '파일접근모드' , encoding='uft-8')  
f.write('데이터')  
f.close()
```

파일 접근 모드	설명
r	읽기모드 - 파일을 읽기만 할 때 사용
w	쓰기모드 - 파일에 내용을 쓸 때 사용. 파일이 존재하는 경우에는 원래있던 내용이 모두 지워지고 새로 저장된다
a	추가모드 - 파일의 마지막에 새로운 내용을 추가 시킬 때 사용

퀴즈

입력된 글자가 숫자가 아니라면 에러 메시지 출력, 입력된 글자가 모두 숫자라면 두 수를 더한 후 출력한다.

```
1 add_except()
```

숫자1 입력 ? 백
입력 데이터가 숫자가 아닙니다.

```
1 add_except()
```

숫자1 입력 ? -90
숫자2 입력 ? 89
 $-90 + 89 = -1$

```
1 add_except()
```

숫자1 입력 ? 100
숫자2 입력 ? 강아지
입력 데이터가 숫자가 아닙니다.

파일 읽고 쓰기

- 파일 생성하기

```
f = open("data/test.txt", 'w')  
f.close()
```

주의사항 -
대상 폴더는 미리 생성되어
있어야 한다.

- 파일을 쓰기 모드로 열어 출력 값 적기

```
f = open( " data/test.txt", 'w')  
for i in range(1, 11):  
    data = "%d번째 줄입니다.\n" % i  
    f.write(data)  
f.close()
```

파일 읽고 쓰기

- 파일에 새로운 내용 추가하기

```
f = open('data/test.txt', 'a')
f.write('-'*50)
for i in range(11,21):
    data = 'Wn %d : Hello Python ' % i
    f.write(data)
f.write('Wn'+ '-'*50)
f.close()
```

파일 읽고 쓰기

- 파일 읽기

`read()`

`readline()` : 첫행만

`readlines()` : 리스트 형태

```
f = open("data/test.txt", 'r')
```

```
data = f.read()
```

```
print(data)
```

```
print(type(data))
```

```
f.close()
```


readline() / readlines()

- readline() 함수 :

f.readline()은 파일의 첫번째 줄을 읽어 출력한다.

```
f = open("data/yesterday.txt", 'r')
```

```
line = f.readline()
```

```
print(line)
```

```
print(type(line))
```

```
f.close()
```

readline() / readlines()

- 모든 라인을 읽어서 화면에 출력한다.

```
f = open('data/Yesterday.txt','r')
```

```
cnt = 1
```

```
while True:
```

```
    line = f.readline()
```

```
    if not line:
```

```
        break
```

```
    print(f'{cnt} : {line}')
```

```
    cnt += 1
```

```
f.close()
```

readline() / readlines()

- readlines() 함수 이용하기

```
f = open('data/Yesterday.txt','r')
```

```
lines = f.readlines()
```

```
for line in lines:
```

```
    print(line)
```

```
f.close()
```

with 문 사용하기

- 파일 Open() 후 close() 명령줄을 삽입하지 않아도 된다.

with open('파일명', '파일 액세스 모드') as 파일변수:
.... 명령문 ...

파일 쓰기

```
with open('data/foo.txt', 'w') as f:  
    f.write("Life is too short, you need python")  
print('파일 쓰기 완료')
```

파일에 내용추가

```
with open('data/foo.txt', 'a') as f:  
    f.write("\n Hello world")  
print('내용 추가 완료')
```

파일 내용읽기

```
with open('data/foo.txt', 'r') as f:  
    data = f.read()  
    print(data)  
print('읽기 완료')
```

퀴즈

'data/coding.txt' 문서를 읽고 코딩이라는 단어가 들어간 어구를 리스트화 한 후 총 갯수를 출력하는 함수를 코딩하여라.

fileread('data/coding.txt', 'utf-8') 함수 호출

파일명 : data/coding.txt

코딩이라는 글자가 들어간 어구 출력

: ['코딩을', '코딩을', '코딩에', '코딩을', '코딩을', '코딩을', '코딩을', '코딩도', '코딩에', '코딩은', '코딩을', '코딩도', '코딩', '코딩도', '코딩을', '코딩을']

총 갯수는? 15 개

퀴즈

readlines()를 이용하여 문서에서 특정 행부터 행까지를 출력하도록 함수 정의하고 출력하도록 코딩하여라

readFirstLines('data/color.txt', 'utf-8', 13, 15) 함수 호출시

파일명 : data/color.txt

13행부터 15행 까지 출력

13 행 : ORANGE를 좋아하는 사람

14 행 : * 심성이 착하고, 다른 사람들과 함께 있기를 좋아하고, 귀가 얇고 충성심이 강하며 술선 수범형.

15 행 : * 인정이 많다. 유쾌한 성격.

퀴즈

readFirstLines('data/Yesterday.txt', 'cp949', 8, 12) 함수 호출시

파일명 : data/Yesterday.txt

총 행 수 : 30

8행부터 12행 까지 출력

8 행 : There's a shadow hanging over me

9 행 : Oh, yesterday came suddenly

10 행 : Why she had to go,

11 행 : I don't know She wouldn't say

퀴즈

n개의 단어를 입력받은 후 output.txt 파일로 저장되도록 함수를 정의하고 호출하여라.

inputWriteFile(5, 'output/output.txt', 'utf-8') 함수 호출시 결과

단어를 입력하세요 ... 장발장
단어를 입력하세요 ... 신데렐라
단어를 입력하세요 ... 콩쥐팥쥐
단어를 입력하세요 ... 라푼젤
단어를 입력하세요 ... 선녀와 나뭇꾼

입력된 단어 리스트는 ['장발장', '신데렐라', '콩쥐팥쥐', '라푼젤', '선녀와 나뭇꾼'] 입니다.

5 개의 단어가 모두 저장되었습니다.

퀴즈

숫자 n 을 입력받아 n 단의 구구단 결과를 특정 문서에 저장되도록 하여라.

출력파일경로 : output/gugu_n.txt

출력 예시

'''

3 단

3 X 1 = 3

...

3 X 9 = 27

'''

CSV 개요

- csv 란?
 - 쉼표로 분리된 데이터
 - comma-separated values
- csv, tsv, dat, xls, json, xml, html => 2차원리스트, 딕셔너리 리스트
- 관련 모듈 : csv, pandas

CSV 개요

연도	제조사	모델	설명	가격
1997	Ford	E350	ac, abs, moon	3000.00
1999	Chevy	Venture "Extended Edition"		4900.00
1999	Chevy	Venture "Extended Edition, Very Large"		5000.00
1996	Jeep	Grand Cherokee	MUST SELL! air, moon roof, loaded	4799.00

```
연도,제조사,모델,설명,가격
1997,Ford,E350,"ac, abs, moon",3000.00
1999,Chevy,"Venture ""Extended Edition""",,4900.00
1999,Chevy,"Venture ""Extended Edition, Very Large""",,5000.00
1996,Jeep,Grand Cherokee,"MUST SELL!air, moon roof, loaded",4799.00
```

CSV 읽기

- CSV 파일 읽기 1 => 중첩 리스트

파일변수 = open('csv파일경로', 'r', encoding='utf-8/cp949')

csv객체변수 = csv.reader(파일변수)

파일변수.close()

- CSV 파일 읽기 2 => 중첩 리스트

with open('csv파일경로', 'r', encoding='utf-8/cp949') as 파일변수

csv객체변수 = csv.reader(파일변수)

CSV 메소드

읽기 관련 (Reader)

- **csv.reader(file, delimiter=',', ...):**
 - 파일 객체를 인자로 받아 CSV 파일의 각 행을 **리스트(list)** 형태로 반환하는 객체 생성.
- **csv.DictReader(file, fieldnames=None, ...):**
 - 각 행을 **딕셔너리(dict)** 형태로 반환.
 - 첫 번째 행을 키(key)로, 이후 행들을 값(value)으로 매핑하여 컬럼명으로 데이터에 접근할 수 있다.

CSV 메소드

쓰기 관련 (Writer)

- **csv.writer(file, delimiter=',', ...):**
 - 파일에 데이터를 쓰기 위한 객체 생성.
 - `writerow()` 메서드로 한 행씩, `writerows()`로 여러 행을 기록할 수 있다.
- **csv.DictWriter(file, fieldnames, ...):**
 - 딕셔너리(dict) 데이터를 CSV 형식으로 쓸때 사용.
 - `fieldnames`를 미리 지정해야 하며, `writerow({'key': 'value'})` 방식으로 데이터 기록이 가능하다

CSV 읽기

- CSV 파일 읽기 1 => 리스트안의 딕셔너리

파일변수 = open('csv파일경로', 'r', encoding='utf-8/cp949')

csv객체변수 = csv.DictReader(파일변수)

data_list = list(csv객체변수)

파일변수.close()

- CSV 파일 읽기 2 => 리스트안의 딕셔너리

with open('csv파일경로', 'r', encoding='utf-8/cp949') as 파일변수

csv객체변수 = csv.DictReader 파일변수)

data_list = list(csv객체변수)

CSV 읽기

파이썬 파일을 기준으로 data 폴더안에 data.csv 파일 읽기

```
import csv, os
print(os.getcwd())
with open('data/data.csv', 'r', encoding='utf-8') as f:
    csv_data = csv.reader(f)
    print(csv_data)

# 2차원 리스트 구조로 변경
data_list = list(csv_data)

# 첫행 리스트만 출력하기
print(data_list[0]) # ['class', 'name', 'kor', 'eng', 'mat', 'bio']

# 1행 제외 나머지 데이터만 출력하기
print('='*50)
for c, n, k, e, m, b in data_list[1:]:
    print(c, n, k, e, m, b)
```


CSV 읽기

```
<_csv.reader object at 0x0000022B214072E0>
['class', 'name', 'kor', 'eng', 'mat', 'bio']
=====
1 adam 67 87 90 98
1 andrew 45 45 56 98
1 ben 95 59 96 88
1 clark 65 94 89 98
1 dan 45 65 78 98
1 noel 78 76 98 89
2 paul 87 67 65 56
2 walter 89 98 78 78
2 oscar 100 78 56 65
2 martin 99 89 87 87
2 hugh 98 45 56 54
2 henry 65 89 87 78
```

CSV 읽기

```
import csv, os
```

파이썬 파일을 기준으로 data 폴더안에 data.csv 파일 읽기

```
file_path = 'data/data.csv'
```

```
with open(file_path, 'r', encoding='utf-8') as f:
```

```
    # DictReader 사용: 첫 행을 자동으로 키(key)로 인식
```

```
    csv_reader = csv.DictReader(f)
```

```
    data_list = list(csv_reader)
```

```
# 헤더 정보 확인 (DictReader에서는 필드명으로 접근 가능)
```

```
print(f"헤더 정보: {csv_reader.fieldnames}")
```

```
# 각 행의 데이터를 딕셔너리 형태로 출력
```

```
print('='*50)
```

```
for row in data_list:
```

```
    print(row['class'], row['name'], row['kor'], row['eng'], row['mat'], row['bio'])
```

CSV 읽기

헤더 정보: ['class', 'name', 'kor', 'eng', 'mat', 'bio']

=====

```
1 adam 67 87 90 98
1 andrew 45 45 56 98
1 ben 95 59 96 88
1 clark 65 94 89 98
1 dan 45 65 78 98
1 noel 78 76 98 89
2 paul 87 67 65 56
2 walter 89 98 78 78
2 oscar 100 78 56 65
2 martin 99 89 87 87
2 hugh 98 45 56 54
2 henry 65 89 87 78
```

퀴즈

- data.csv 파일의 데이터를 아래와 같이 출력하여라

class	name	kor	eng	mat	bio
1	adam	67	87	90	98
1	andrew	45	45	56	98
1	ben	95	59	96	88
1	clark	65	94	89	98
1	dan	45	65	78	98
1	noel	78	76	98	89
2	paul	87	67	65	56
2	walter	89	98	78	78
2	oscar	100	78	56	65
2	martin	99	89	87	87
2	hugh	98	45	56	54
2	henry	65	89	87	78

퀴즈

- 국어(kor) 과목의 총점, 평균, 최고점, 최저점을 출력하여라.
sum(), min(), max(), len() 리스트 관련 함수 활용

```
국어 점수 리스트 : [67, 45, 95, 65, 45, 78, 87, 89, 100, 99, 98, 65]  
총점 : 933, 평균 : 77.75  
최고점 : 100, 최저점 : 45
```

- 국어(kor) 과목 점수를 80점 이상 받은 학생의 이름과 점수를 출력하여라.

```
ben   : 95 점  
paul  : 87 점  
walter : 89 점  
oscar : 100 점  
martin : 99 점  
hugh  : 98 점
```

퀴즈

- wages.csv 파일에서 아래의 조건에 해당하는 데이터만 출력하여라
race : hispanic , gender : female, age <= 30
총 데이터 개수도 함께 출력한다.

```
=====
race      age    ed    height  gender    earn
=====
hispanic  25     12    60.61   female    18497.8538930071
hispanic  26     12    60.75   female    16905.0220799872
hispanic  27     16    64.95   female    28032.1868635186
hispanic  25     11    63.94   female    24860.3637743202
hispanic  27     16    61.15   female    984.500213531472
hispanic  23     12    60.56   female    21659.5418912689
hispanic  30     17    65.68   female    32823.4149412493
hispanic  27     9     63.01   female    998.767187740165
hispanic  26     13    63.81   female    15309.4702056486
hispanic  25     12    65.86   female    3375.81930734894
```

전체 데이터 갯수는 ? 10

CSV 파일 쓰기

- csv 파일 쓰기1

파일변수 = open('csv파일경로', 'w', encoding='utf-8/cp949', newline='')

csv객체변수 = csv.writer(파일변수)

csv객체변수.writerow(리스트/튜플...)

파일변수.close()

CSV 파일 쓰기

- csv 파일 쓰기2

with open('csv파일경로', 'w', encoding='utf-8/cp949', newline='') as 파일변수:

 csv객체변수 = csv.writer(파일변수)

 csv객체변수.writerow(리스트/튜플...)

CSV 파일 쓰기

2차원 리스트

```
data_list = [ ['이름','주소','전화번호'],  
              ['김영희','부산시','010-6374-90874'],  
              ['홍길동','춘천시','010-5463-9403'],  
              ['성은희','서울시','010-4646-9403'] ]
```

2차원 리스트 => csv 파일로 저장
현재폴더를 기준으로 output 폴더 생성
os.mkdir(폴더명)

newline="" 공백줄 없이 삽입하는 옵션

with open('output/address1.csv', 'w', encoding='utf-8', newline='') as f:

csv 객체 생성

csv_data = csv.writer(f)

1~2행만 쓰기

csv_data.writerow(data_list[0])

csv_data.writerow(data_list[1])

 address1.csv

이름, 주소, 전화번호

김영희, 부산시, 010-6374-90874

CSV 파일 쓰기

```
# 2차원 리스트 전체 데이터 csv 파일로 저장하기 1 - for문 + writerow(리스트/튜플...)
with open('output/address2.csv', 'w', encoding='utf-8', newline='') as f:
    # csv 객체 생성
    csv_data = csv.writer(f)
    # 전체쓰기
    for row in data_list:
        csv_data.writerow(row)
```

 address2.csv

이름, 주소, 전화번호

김영희, 부산시, 010-6374-90874

홍길동, 춘천시, 010-5463-9403

성은희, 서울시, 010-4646-9403

CSV 파일 쓰기

```
# 2차원 리스트 전체 데이터 csv 파일로 저장하기 2 - writerows(2차원리스트)
with open('output/address3.csv', 'w', encoding='utf-8', newline='') as f:
    # csv 객체 생성
    csv_data = csv.writer(f)
    # 1행만 삽입
    csv_data.writerow(['주소'])

    # 전체쓰기
    csv_data.writerows(data_list)
```

📄 address3.csv

```
주소
이름, 주소, 전화번호
김영희, 부산시, 010-6374-90874
홍길동, 춘천시, 010-5463-9403
성은희, 서울시, 010-4646-9403
```

CSV 파일 쓰기

```
# 제목행과 관련된 필드 쓰기  
csv_data.writeheader()
```

```
# 전체 쓰기  
for dict in dict_list:  
    csv_data.writerow(dict)
```

 address4.csv

```
name,address,mobile  
김영희,부산시,010-6374-90874  
홍길동,춘천시,010-5463-9403  
성은희,서울시,010-4646-9403
```

CSV 파일 쓰기

```
# writerows() 함수를 이용한 전체 쓰기
with open('output/address5.csv', 'w', encoding='utf-8', newline='') as f:
    # 키값으로 구성된 리스트 생성
    field_list = ['name', 'address', 'mobile']
    csv_data = csv.DictWriter(f, fieldnames=field_list)
    csv_data.writeheader()
    csv_data.writerows(dict_list)
```

 address5.csv

```
name,address,mobile
김영희,부산시,010-6374-90874
홍길동,춘천시,010-5463-9403
성은희,서울시,010-4646-9403
```

퀴즈

이름, 나이, 직업을 입력받은 후 quiz_data.csv 파일로 데이터를 저장하도록 프로그래밍 하여라.

- 필드 제목은 이름, 나이, 직업으로 한다
- y를 입력하면 계속 입력, n을 입력하면 프로그램을 종료시킨다.

퀴즈

CSV 파일에 저장할 데이터를 입력하세요.

이름: 홍길동

나이: 25

직업: 개발자

데이터가 CSV 파일에 저장되었습니다!

더 입력하시겠습니까? (y/n): y

CSV 파일에 저장할 데이터를 입력하세요.

이름: 김철수

나이: 30

직업: 데이터 분석가

데이터가 CSV 파일에 저장되었습니다!

더 입력하시겠습니까? (y/n): n

프로그램을 종료합니다.

workspace20250220 > quiz_data.csv

1	이름, 나이, 직업
2	홍길동, 25, 개발자
3	김철수, 30, 데이터분석가
4	